Step 1: Project Setup

📁 Create a folder

```
todo-app/
 └── index.html
```

Open `index.html` in VS Code or any editor.

□ Step 2: Basic HTML Structure

Paste this skeleton:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Todo App</title>
</head>
<body>

  <h2>Todo App</h2>

  <input type="text" id="todo-input" placeholder="Enter task">
  <button onclick="addTodo()">Add</button>

  <ul id="todo-list"></ul>

</body>
</html>
```

🖼 UI Structure (Conceptual View)

```
+----------------------+
|      Todo App        |
+----------------------+
| [ Enter task      ]  |  + Add button
|                      |
|  • Task 1   ✓ ✕      |
|  • Task 2   ✓ ✕      |
+----------------------+
```

🎨 Step 3: Add CSS Styling

Inside `<head>` add:

```
<style>
```

```
  body {
    font-family: Arial;
    background: #f4f4f4;
    display: flex;
    justify-content: center;
    margin-top: 50px;
  }

  .app {
    background: white;
    padding: 20px;
    width: 300px;
    border-radius: 10px;
  }

  li {
    display: flex;
    justify-content: space-between;
    margin-top: 10px;
  }

  .completed {
    text-decoration: line-through;
    color: gray;
  }
</style>
```

🖼 Styled Layout

```
        (Centered Card)

     ┌──────────────┐
     │   Todo App   │
     ├──────────────┤
     │ [ Task input ]│
     │              │
     │  Task ✓ ✕    │
     │  Task ✓ ✕    │
     └──────────────┘
```

⚙ Step 4: Add JavaScript Logic

Add this before `</body>`:

```
<script>
  let todos = JSON.parse(localStorage.getItem('todos')) || [];

  function saveTodos() {
    localStorage.setItem('todos', JSON.stringify(todos));
  }
</script>
```

□ Concept

```
User Action → Update Array → Save to localStorage
```

Example storage:

```
[
  { "text": "Study JS", "completed": false }
]
```

➕ Step 5: Add Todo Function

```
function addTodo() {
  const input = document.getElementById('todo-input');
  const text = input.value.trim();

  if (text === '') return;

  todos.push({ text, completed: false });

  input.value = '';
  saveTodos();
  renderTodos();
}
```

🖼 Flow

```
Input → Click Add → Push into Array → Save → Render
```

🗑 Step 6: Delete Function

```
function deleteTodo(index) {
  todos.splice(index, 1);
  saveTodos();
  renderTodos();
}
```

🖼 Flow

```
Click ✕ → Remove from Array → Update UI
```

✅ Step 7: Toggle Complete

```
function toggleComplete(index) {
  todos[index].completed = !todos[index].completed;
  saveTodos();
  renderTodos();
```

```
}
```

🖼 Flow

```
Click ✓ → completed = true/false → UI update
```

🔄 Step 8: Render Function (Core UI Engine)

```
function renderTodos() {
  const list = document.getElementById('todo-list');
  list.innerHTML = '';

  todos.forEach((todo, index) => {
    const li = document.createElement('li');

    if (todo.completed) li.classList.add('completed');

    li.innerHTML = `
      <span>${todo.text}</span>
      <div>
        <button onclick="toggleComplete(${index})">✓</button>
        <button onclick="deleteTodo(${index})">✕</button>
      </div>
    `;

    list.appendChild(li);
  });
}
```

🖼 Rendering Logic

```
Array → Loop → Create <li> → Insert into DOM
```

🚀 Step 9: Initial Load

```
renderTodos();
```

🖼 App Lifecycle

```
Page Load
   ↓
Read localStorage
   ↓
Render UI
```

💾 Step 10: localStorage Working

Data lifecycle:

```
Add Task
   ↓
todos[] updated
   ↓
JSON.stringify()
   ↓
localStorage.setItem()
```

Retrieval:

```
localStorage.getItem()
   ↓
JSON.parse()
   ↓
todos[]
```

□ Final Output Behavior

✓ Add task
✓ Mark complete
✓ Delete task
✓ Data saved after refresh

📌 What You Built (Architecture)

```
UI Layer (HTML + CSS)
        ↓
Event Layer (onclick)
        ↓
Logic Layer (JS functions)
        ↓
Storage Layer (localStorage)
```